

Data Structures Lab – Assignment 7

March 25, 2026

Assignment: Stack Operations

Objective:

Understand the fundamental operations of a stack (push, pop, peek) and their implementation and apply stack concepts to solve real-world and computational problems such as **expression evaluation**, **Next Greater Element**, and designing an efficient **Min Stack**.

1. Palindrome Check Using Stack

Write a program using a stack to determine whether a given string is a palindrome or not. The program should push all characters of the string onto the stack and then compare them while popping.

Sample Input:

String = madam

Sample Output:

The given string is a Palindrome

2. Infix to Postfix Conversion Using Stack

Write an algorithm and program to convert a given infix expression into its equivalent postfix expression using a stack. Consider operator precedence and associativity rules.

Sample Input:

Infix Expression: $(A + B) * C$

Sample Output:

Postfix Expression: $A B + C *$

3. Infix to Prefix Conversion Using Stack

Write an algorithm and program to convert a given infix expression into its equivalent prefix expression using a stack. Clearly show the steps involved, including reversal of the expression and handling of parentheses.

Sample Input:

Infix Expression: $(A + B) * C$

Sample Output:

Prefix Expression: * + A B C

4. Expression Evaluation Using Stack

Write a program to evaluate a given arithmetic expression in **postfix notation** using a stack. The program should correctly handle operators such as +, -, *, and /.

Sample Input:

Postfix Expression: 2 3 1 * + 9 -

Sample Output:

Result: -4

5. Next Greater Element

Given an array of integers, write a program to find the **Next Greater Element (NGE)** for each element in the array using a stack. The Next Greater Element for an element x is the first greater element on its right. If no such element exists, output -1.

Sample Input:

N = 4

Array = [4, 5, 2, 10]

Sample Output:

Next Greater Elements = [5, 10, 10, -1]

6. Design a Min Stack

Design and implement a stack data structure that supports the following operations in $O(1)$ time:

- push(x): Insert an element into the stack
- pop(): Remove the top element from the stack
- top(): Return the top element
- getMin(): Retrieve the minimum element in the stack

Write a program to perform a sequence of operations and display the output.

Sample Input 1:

push(5), push(3), push(7), getMin(), pop(), getMin()

Sample Output 1:

3

3

Sample Input 2:

Operations:

push(10), push(20), push(5), getMin(), push(15), getMin(),
push(3), getMin(), pop(), getMin(), pop(), top(), getMin()

Sample Output 2:

Operation	Stack	Minimum
push(10)	[10]	10
push(20)	[10, 20]	10
push(5)	[10, 20, 5]	5
getMin()	—	5
push(15)	[10, 20, 5, 15]	5
getMin()	—	5
push(3)	[10, 20, 5, 15, 3]	3
getMin()	—	3
pop()	[10, 20, 5, 15]	5
getMin()	—	5
pop()	[10, 20, 5]	5
top()	—	5
getMin()	—	5

Sample Output:

5
5
3
5
5
5